

What is JSON?

JSON stands for

JavaScript Object Notation

Despite the name, it is language-independent.

Today almost every application exchanges data using JSON.

Example

```
{
  "name": "Aadya",
  "age": 20,
  "cgpa": 7.97,
  "ethereal": true
}
```

Suppose you're storing user preferences.

Different users may have different settings.

User A

```
{
  "theme": "dark",
  "language": "English"
}
```

User B

```
{
  "theme": "light",
  "fontSize": 18,
  "notifications": false,
  "language": "Hindi"
}
```

Creating separate columns for every possible preference would be impractical. JSON allows flexible storage.

JSON Data Type

MySQL provides a native JSON datatype.

Example

```
CREATE TABLE Users(  
    UserID INT PRIMARY KEY,  
    Preferences JSON  
);
```

Insert

```
INSERT INTO Users VALUES  
(1, '{  
    "theme": "dark",  
    "language": "English"  
}')
```

MySQL stores JSON in an optimized binary format internally.

Benefits:

- Faster retrieval.
- Faster parsing.
- Automatic validation.
- More efficient storage than plain text.

Rules

Keys:

- are strings,

- must be enclosed in double quotes.

Values may be

- string,
- number,
- Boolean,
- null,
- object,
- array.

Arrays may also contain objects.

Example

```
{
  "projects":[
    {
      "name":"Hospital System",
      "year":2025
    },
    {
      "name":"MediSync",
      "year":2026
    }
  ]
}
```

Nested JSON

Objects can contain other objects.

Example

```
{
  "name": "Aadya",
  "address": {
    "city": "Gwalior",
    "state": "Madha pradeshMySQL stores JSON efficiently in binary form.
You don't need to manually serialize or deserialize it when storing or
retrieving.",
    "country": "India"
  }
}
```

Querying JSON

One powerful feature of MySQL is querying values inside JSON documents.
Suppose

```
{
  "theme": "dark",
  "language": "English"
}
```

stored in

Preferences

Retrieve theme

```
SELECT JSON_EXTRACT(Preferences, '$.theme')FROM Users;
```

Output

```
"dark"
```

\$.theme means:

- \$ → root of the JSON document.
- .theme → value of the theme key.

Another example

```
{
  "address":{
    "city":"Jaipur"
  }
}
```

Retrieve city

```
SELECT JSON_EXTRACT(Preferences, '$.address.city')FROM Users;
```

You can also use the shorthand operators in MySQL:

```
SELECT Preferences->'$.theme'FROM Users;
```

Returns the unquoted SQL string (dark instead of "dark")

Indexing Basics

Suppose your application frequently searches by:

theme

inside JSON.

Without indexing,

MySQL scans every row.

Instead,

you can create a generated column that extracts the JSON value and then index that generated column.

Example

```
ALTER TABLE UsersADD COLUMN Theme VARCHAR(20)
GENERATED ALWAYS AS (Preferences->>'$.theme') STORED;CREATE INDEX idx_themeON
Users(Theme);
```

API Responses

Many REST APIs return JSON.

Example

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "Aadya"
  }
}
```

Storing the raw response as JSON can be useful for debugging or auditing.

Advantages & Disadvantages

Advantages

- Flexible schema.
- Built-in validation.
- Efficient binary storage.
- Easy integration with web APIs.
- Supports nested structures.
- Powerful querying functions.

Disadvantages

- Complex JSON queries can be slower than simple column lookups.
- Excessive use can make data harder to analyze.

JSON vs Normalized Tables

Suppose

Student

StudentID	Name
1	Aadya

Preferences

StudentID	Theme	Language
1	Dark	English

This is normalized.

Alternative

Student

StudentID	Name	Preferences
1	Aadya	{"theme":"dark","language":"English"}

Both work.

Which is better?

Depends.

Use JSON when:

- attributes vary between rows,
- relationships aren't important.

Use normalized tables when:

- relationships matter,
- joins are common,
- data must be consistent,

When should JSON be avoided?

Avoid it when:

- the data has clear relational structure,
- you frequently join, group, or aggregate on the fields,